Lecture 12 - SQL and MySQL

Thursday, October 29, 2026 11:48 PM

Lecture 12 - Doctor homework review, SQL and MySQL

Setup checklist

MySQL Server is the database process. Workbench is a separate SQL client. The existing Hospital application uses Java 25, Maven, JdbcTemplate and HTTP port 8081. MySQL commonly listens on port 3306; these ports have different purposes.

MySQL Server must be running, the SQL client connection must work, and the correct Hospital project must be open. MySQL credentials belong in the local Run Configuration. Existing H2 files can remain as a separate checkpoint.

```
SELECT VERSION() AS server_version;
SELECT CURRENT_USER() AS connected_account;
SELECT @@port AS server_port;
SELECT 1 AS connection_ok;
```

0. Homework: Doctor workload report and one balancing step

Workload means the number of patients currently assigned to a doctor. The application stores assignments in patients.doctor_id; Doctor.patientList is assembled from current database rows.

GET /doctors/workload returns every doctor's ID, name and patient count, including zero-patient doctors. Sort by count descending, then doctor ID ascending. No doctors returns 200 with [].

POST /doctors/rebalance performs one step: find the busiest and least busy doctors; if their counts differ by more than one, move the lowest-ID patient from busiest to least busy. Break doctor-count ties using the lowest doctor ID. Ignore unassigned patients. With fewer than two doctors, or a gap at most one, return moved=false, a message and null IDs. A successful step returns moved=true with the moved patient and source/target doctor IDs.

```
[
  {"doctorId":1,"doctorName":"Dr Watson","patientCount":5},
  {"doctorId":2,"doctorName":"Dr Banner","patientCount":1},
  {"doctorId":3,"doctorName":"Dr Strange","patientCount":0}
]
```

```
Before: Watson 5, Banner 1, Strange 0
One POST → move lowest-ID Watson patient to Strange
After:  Watson 4, Banner 1, Strange 1
```

Prediction questions

Should counts 2/1 trigger a move? With doctor counts 1:2, 2:2, 3:0, 4:0, which source and target are chosen? How is the selected patient made deterministic? Does one request fully redistribute every patient?

Response models

Create DoctorWorkload.java and RebalanceResult.java under com.accio.springProject.Doctors.Model. These are response models, not additional database tables. Long fields can hold null when no patient moves.

```
package com.accio.springProject.Doctors.Model;

public class DoctorWorkload {
    private final long doctorId;
    private final String doctorName;
    private final int patientCount;

    public DoctorWorkload(long doctorId, String doctorName, int patientCount) {
        this.doctorId = doctorId;
        this.doctorName = doctorName;
        this.patientCount = patientCount;
    }

    public long getDoctorId() { return doctorId; }
    public String getDoctorName() { return doctorName; }
    public int getPatientCount() { return patientCount; }
}
```

```
package com.accio.springProject.Doctors.Model;

public class RebalanceResult {
    private final boolean moved;
    private final String message;
    private final Long patientId;
    private final Long fromDoctorId;
    private final Long toDoctorId;

    public RebalanceResult(boolean moved, String message,
            Long patientId, Long fromDoctorId, Long toDoctorId) {
        this.moved = moved;
        this.message = message;
        this.patientId = patientId;
        this.fromDoctorId = fromDoctorId;
        this.toDoctorId = toDoctorId;
    }

    public boolean isMoved() { return moved; }
    public String getMessage() { return message; }
    public Long getPatientId() { return patientId; }
    public Long getFromDoctorId() { return fromDoctorId; }
    public Long getToDoctorId() { return toDoctorId; }
}
```

Repository additions

In the existing Doctors/Repository/DoctorRepository, retain doctorRowMapper, patientRowMapper, findById and assignPatients. Add findAll, then replace findPatientsByDoctorId with the ordered version. ORDER BY id makes the first returned patient the lowest-ID patient.

```
public List<Doctor> findAll() {
    return jdbc.query(
            "SELECT id, name FROM doctors ORDER BY id", doctorRowMapper);
}
```

```
public List<Patient> findPatientsByDoctorId(long doctorId) {
    return jdbc.query(
            "SELECT id, name, age FROM patients WHERE doctor_id = ? ORDER BY id",
            patientRowMapper, doctorId);
}
```

Workload service and endpoint

Add these imports to DoctorService, retaining its existing Doctor, Patient and List imports and its constructor:

```
import com.accio.springProject.Doctors.Model.DoctorWorkload;
import com.accio.springProject.Doctors.Model.RebalanceResult;
import java.util.ArrayList;
import java.util.Comparator;
```

```
public List<DoctorWorkload> getWorkload() {
    List<DoctorWorkload> result = new ArrayList<>();
    for (Doctor doctor : repository.findAll()) {
        int count = repository.findPatientsByDoctorId(doctor.getId()).size();
        result.add(new DoctorWorkload(doctor.getId(), doctor.getName(), count));
    }

    result.sort(Comparator.comparingInt(DoctorWorkload::getPatientCount)
            .reversed()
            .thenComparingLong(DoctorWorkload::getDoctorId));
    return result;
}
```

In DoctorController, import DoctorWorkload and java.util.List. Retain ResponseEntity and the /doctors class-level mapping.

```
@GetMapping("/workload")
public ResponseEntity<List<DoctorWorkload>> getWorkload() {
    return ResponseEntity.ok(service.getWorkload());
}
```

Balancing service and endpoint

The report ordering identifies the busiest doctor. The least-busy doctor needs a separate lowest-ID tie-break; choosing the last sorted element would select the highest ID among tied minimum counts.

```
public RebalanceResult rebalance() {
    List<DoctorWorkload> workloads = getWorkload();
    if (workloads.size() < 2) {
        return new RebalanceResult(false, "At least two doctors are needed",
                null, null, null);
    }

    DoctorWorkload busiest = workloads.get(0);
    DoctorWorkload leastBusy = workloads.get(0);
    for (DoctorWorkload current : workloads) {
        if (current.getPatientCount() < leastBusy.getPatientCount()
                || (current.getPatientCount() == leastBusy.getPatientCount()
                && current.getDoctorId() < leastBusy.getDoctorId())) {
            leastBusy = current;
        }
    }

    if (busiest.getPatientCount() - leastBusy.getPatientCount() <= 1) {
        return new RebalanceResult(false, "No balancing needed",
                null, null, null);
    }

    List<Patient> patients =
            repository.findPatientsByDoctorId(busiest.getDoctorId());
    if (patients.isEmpty()) {
        return new RebalanceResult(false, "Workload changed; try again",
                null, null, null);
    }

    long patientId = patients.get(0).getId();
    boolean updated = repository.assignPatients(leastBusy.getDoctorId(), patientId);
    if (!updated) {
        return new RebalanceResult(false, "Patient no longer exists; try again",
                null, null, null);
    }

    return new RebalanceResult(true, "One patient reassigned",
            patientId, busiest.getDoctorId(), leastBusy.getDoctorId());
}
```

```
Read workloads → choose busiest and least busy → gap > 1?
                                        no → return moved=false
                                        yes
                                         ↓
Read source patients ordered by ID → move first one → return IDs
```

In DoctorController, import RebalanceResult and add the POST endpoint. It takes no request body.

```
@PostMapping("/rebalance")
public ResponseEntity<RebalanceResult> rebalance() {
    return ResponseEntity.ok(service.rebalance());
}
```

Behaviour checks

```
GET /doctors/workload
→ verify counts and descending-count/ascending-ID order

POST /doctors/rebalance (no body)
→ moved=true; reports actual source, target and lowest source patient ID

GET /doctors/workload
→ source loses exactly one; target gains exactly one

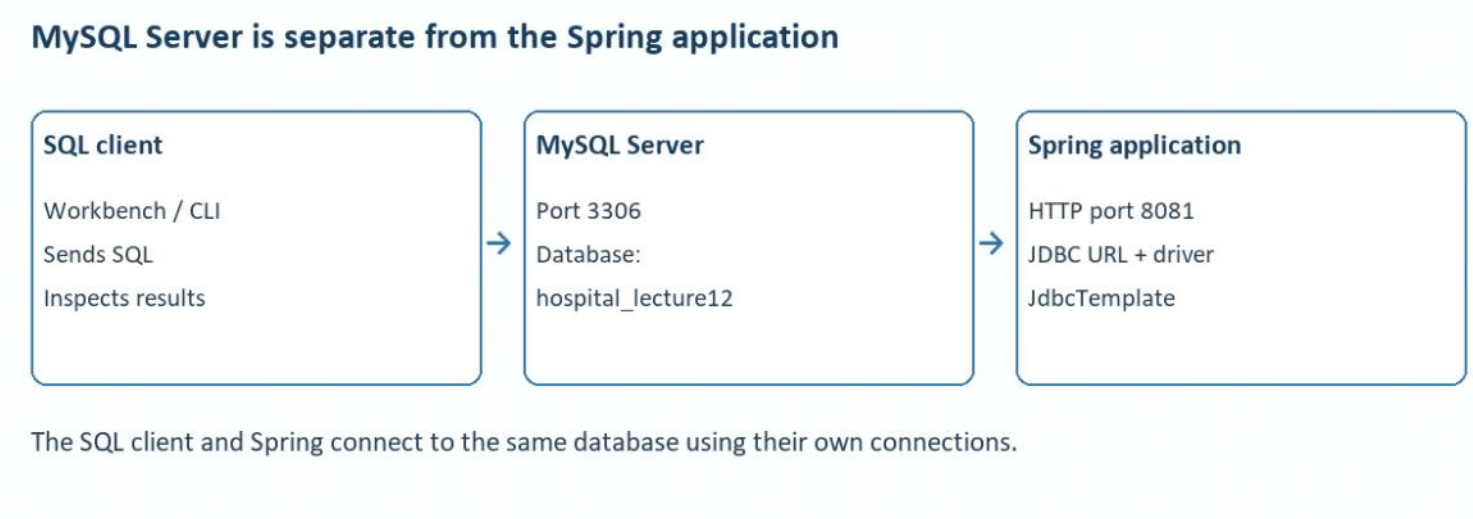
GET /doctors/{fromDoctorId}/patients
GET /doctors/{toDoctorId}/patients
→ moved patient absent from source and present at target

Restart using the same H2 file, then repeat GET
→ changed assignment remains
```

Check zero doctors, one doctor, all-zero counts, a count difference of one, tied source/target counts and persistence after restart. A move preserves the total assigned-patient count and never moves unassigned patients. This exercise evaluates sequential requests; the read/choose/update sequence does not provide concurrent-request protection.

1. SQL, MySQL and the class database

SQL is a query language. MySQL and H2 are database systems that implement SQL with some syntax differences. MySQL Server runs separately from Spring; Workbench and Spring make their own connections to it.



CREATE DATABASE creates a named database; USE selects it for the current SQL-client session. Spring selects its database through the JDBC URL instead. utf8mb4 supports the full Unicode range, including multilingual text and emojis; a character can require up to four bytes.

```
CREATE DATABASE IF NOT EXISTS hospital_lecture12
    CHARACTER SET utf8mb4;

USE hospital_lecture12;
SELECT DATABASE();
```

Tables and data types

Create doctors before patients because the patient table references it. BIGINT stores long IDs, INT stores age, and VARCHAR(255) stores variable-length text within its limit. AUTO_INCREMENT lets MySQL allocate new patient IDs.

```
CREATE TABLE IF NOT EXISTS doctors (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255) NOT NULL
);

CREATE TABLE IF NOT EXISTS patients (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(255) NOT NULL,
    age INT NOT NULL,
    doctor_id BIGINT NULL,
    CONSTRAINT chk_patient_age CHECK (age BETWEEN 0 AND 120),
    CONSTRAINT fk_patients_doctor
        FOREIGN KEY (doctor_id) REFERENCES doctors(id)
);



SELECT VERSION();
SHOW TABLES;
DESCRIBE patients;
```

Seed data

Run the fixed-ID INSERT statements once in a fresh class database. Inspect existing rows first if the database already contains data. This seed does not copy H2 data. Duplicate patient names are permitted; patient IDs identify distinct patients.

```
INSERT INTO doctors (id, name)
VALUES (1, 'Dr Watson'), (2, 'Dr Banner'), (3, 'Dr Strange');

INSERT INTO patients (id, name, age, doctor_id)
VALUES
    (1, 'James', 20, 1),
    (2, 'Jean', 30, 1),
    (3, 'Jean', 40, 2),
    (4, 'Ron', 17, NULL),
    (5, 'Luna', 30, 2),
    (6, 'John', 60, NULL);
```

Lecture 12 seed: doctors and patients

doctors

id	name
1	Dr Watson
2	Dr Banner
3	Dr Strange

patients

id	name	age	doctor_id
1	James	20	1
2	Jean	30	1
3	Jean	40	2
4	Ron	17	NULL
5	Luna	30	2
6	John	60	NULL

patients.doctor_id → doctors.id NULL doctor_id = unassigned patient

All Hospital SQL examples below use this six-patient seed. The self-join example uses a separate

temporary employee table. The Doctor workload homework can be completed against the existing H2 checkpoint before switching to this MySQL dataset.

2. Keys, constraints and NULL

A primary key uniquely identifies a row and cannot be NULL. A table has one primary-key constraint, which may contain multiple columns. A unique constraint enforces another uniqueness rule; MySQL permits multiple NULL entries in a nullable UNIQUE column.

A foreign key enforces valid stored references. Here, every non-null patients.doctor_id must match an existing doctors.id. Several patients may reference one doctor. NOT NULL prevents missing SQL values; it does not reject blank strings. CHECK enforces a condition such as an age range.

A candidate key is a minimal unique identifier. A superkey may include unnecessary columns. A composite key uses several columns. In this example, id is a candidate key and (id, name) is a superkey; patient name alone is not unique.

NULL represents an absent/unknown value. It is different from zero, an empty string and the text 'null'. Which of these queries finds unassigned patients?

```
SELECT id, name FROM patients WHERE doctor_id = NULL;
SELECT id, name FROM patients WHERE doctor_id IS NULL;
```

The comparison = NULL is unknown and does not pass WHERE. IS NULL finds Ron and John, IDs 4 and 6. IS NOT NULL finds assigned patients.

Constraint check

What happens to the existing assignment when the requested doctor ID does not exist?

```
UPDATE patients SET doctor_id = 999999 WHERE id = 1;
SELECT id, name, doctor_id FROM patients WHERE id = 1;
```

The foreign-key constraint rejects the UPDATE, and patient 1 remains assigned to doctor 1. Application validation and database constraints serve different purposes: readable request validation and stored-data rules.

3. Filtering, ordering and limiting

Predict the returned IDs before running each query. BETWEEN includes both boundaries. LIKE 'J%' matches names starting with J. ORDER BY defines ordering; LIMIT restricts the number of rows but does not establish an order by itself.

```
SELECT id, name, age
FROM patients
WHERE age BETWEEN 18 AND 40
ORDER BY age, id;

SELECT id, name FROM patients
WHERE name LIKE 'J%'
ORDER BY id;

SELECT id, name, age FROM patients
ORDER BY age DESC, id
LIMIT 2;
```

Results: the age-range query returns IDs 1, 2, 5, 3; the name query returns James, Jean, Jean and John (IDs 1, 2, 3, 6); the oldest two are John/6 and Jean/3. Text comparison case behaviour depends on collation.

DELETE removes rows and can use WHERE. TRUNCATE empties the table without WHERE and resets MySQL's auto-increment counter. DROP removes the table definition and its data. MySQL DROP/TRUNCATE have implicit-commit behaviour. These are comparisons; no DROP/TRUNCATE is needed on the class Hospital tables.

4. JOINs: match rows, then produce a result

A JOIN combines rows according to its ON condition. Column names need not be identical, and a declared foreign key is not required. In this schema p.doctor_id = d.id follows the assignment relationship; p.id = d.id is valid SQL but compares unrelated entity identifiers.

d and p are table aliases. AS names an output column. ON defines the match. SELECT chooses displayed columns; WHERE can inspect available columns that SELECT does not display.

INNER JOIN — matching pairs

Which patients and doctors have no match? Can one doctor appear more than once?

```
SELECT d.id AS doctor_id, d.name AS doctor,
       p.id AS patient_id, p.name AS patient
FROM doctors d
INNER JOIN patients p ON p.doctor_id = d.id
ORDER BY d.id, p.id;
```



INNER JOIN: only matching pairs

Both sides must satisfy the ON condition.

Seeded doctors

id	doctor
1	Dr Watson
2	Dr Banner
3	Dr Strange

Seeded patients

id	patient	age	doctor_id
1	James	20	1
2	Jean	30	1
3	Jean	40	2
4	Ron	17	NULL
5	Luna	30	2
6	John	60	NULL

ON p.doctor_id = d.id

Query result: 4 rows

doctor_id	doctor	patient_id	patient
1	Dr Watson	1	James
1	Dr Watson	2	Jean
2	Dr Banner	3	Jean
2	Dr Banner	5	Luna

Patient IDs 4 and 6 have no matching doctor. Doctor 3 has no matching patient. None of these unmatched rows appear.

LEFT JOIN — preserve doctors

Which table is on the left? What values fill the patient columns for an unmatched doctor?

```
SELECT d.id AS doctor_id, d.name AS doctor,
       p.id AS patient_id, p.name AS patient
FROM doctors d
LEFT JOIN patients p ON p.doctor_id = d.id
ORDER BY d.id, p.id;
```

LEFT JOIN: preserve doctors

FROM doctors d LEFT JOIN patients p preserves the doctors table.

Seeded doctors

id	doctor
1	Dr Watson
2	Dr Banner
3	Dr Strange

Seeded patients

id	patient	age	doctor_id
1	James	20	1
2	Jean	30	1
3	Jean	40	2
4	Ron	17	NULL
5	Luna	30	2
6	John	60	NULL

ON p.doctor_id = d.id

Query result: 5 rows

doctor_id	doctor	patient_id	patient
1	Dr Watson	1	James
1	Dr Watson	2	Jean
2	Dr Banner	3	Jean
2	Dr Banner	5	Luna
3	Dr Strange	NULL	NULL

Dr Strange stays with NULL patient columns. Dr Watson and Dr Banner each match two patients, so each appears twice.

RIGHT JOIN — preserve patients

Here patients is the right table. Do Ron and John remain? Does Dr Strange remain?

```
SELECT d.id AS doctor_id, d.name AS doctor,
       p.id AS patient_id, p.name AS patient
FROM doctors d
RIGHT JOIN patients p ON p.doctor_id = d.id
ORDER BY d.id, p.id;
```

RIGHT JOIN: preserve patients

FROM doctors d RIGHT JOIN patients p preserves the patients table.

Seeded doctors

id	doctor
1	Dr Watson
2	Dr Banner
3	Dr Strange

Seeded patients

id	patient	age	doctor_id
1	James	20	1
2	Jean	30	1
3	Jean	40	2
4	Ron	17	NULL
5	Luna	30	2
6	John	60	NULL

ON p.doctor_id = d.id

Query result: 6 rows

doctor_id	doctor	patient_id	patient
NULL	NULL	4	Ron
NULL	NULL	6	John
1	Dr Watson	1	James
1	Dr Watson	2	Jean
2	Dr Banner	3	Jean
2	Dr Banner	5	Luna

Ron and John remain with NULL doctor columns. Dr Strange is absent because no patient matches doctor 3. NULL doctor IDs sort first here.

Full outer result in MySQL

A full outer result includes matching pairs and unmatched rows from both sides. MySQL 8.4 does not support direct FULL OUTER JOIN syntax. The following query keeps the LEFT JOIN rows and adds only unmatched patients.

UNION ALL stacks the two compatible results. WHERE d.id IS NULL prevents matched pairs from being appended a second time. The final ORDER BY uses output aliases.

```
SELECT d.id AS doctor_id, d.name AS doctor,
       p.id AS patient_id, p.name AS patient
FROM doctors d
LEFT JOIN patients p ON p.doctor_id = d.id

UNION ALL

SELECT d.id AS doctor_id, d.name AS doctor,
       p.id AS patient_id, p.name AS patient
FROM doctors d
RIGHT JOIN patients p ON p.doctor_id = d.id
WHERE d.id IS NULL
ORDER BY doctor_id, patient_id;
```

Full outer result in MySQL: LEFT JOIN + unmatched RIGHT rows

UNION ALL appends only right-side rows with no doctor match.

Seeded doctors

id	doctor
1	Dr Watson
2	Dr Banner
3	Dr Strange

Seeded patients

id	patient	age	doctor_id
1	James	20	1
2	Jean	30	1
3	Jean	40	2
4	Ron	17	NULL
5	Luna	30	2
6	John	60	NULL

ON p.doctor_id = d.id

Query result: 7 rows

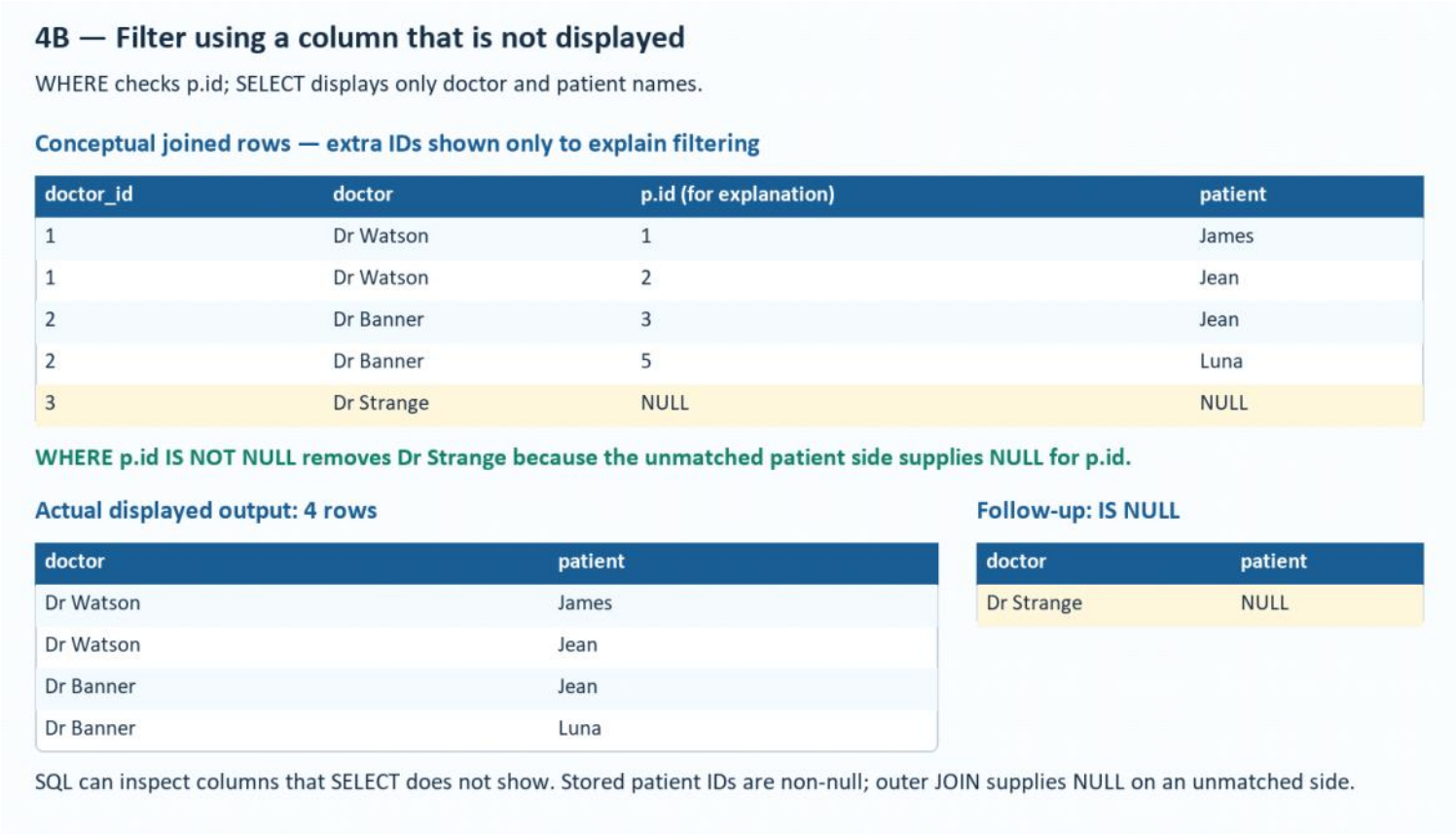
doctor_id	doctor	patient_id	patient
NULL	NULL	4	Ron
NULL	NULL	6	John
1	Dr Watson	1	James
1	Dr Watson	2	Jean
2	Dr Banner	3	Jean
2	Dr Banner	5	Luna
3	Dr Strange	NULL	NULL

Seven rows: four matched pairs + Dr Strange without a patient + Ron and John without a doctor. WHERE d.id IS NULL prevents matched pairs from being added twice.

4B. Filter with an undisplayed column

Can WHERE use p.id when SELECT displays only names? Does LEFT JOIN guarantee that every doctor survives a later WHERE condition?

```
SELECT d.name AS doctor, p.name AS patient
FROM doctors d
LEFT JOIN patients p ON p.doctor_id = d.id
WHERE p.id IS NOT NULL
ORDER BY d.id, p.id;
```



The intermediate p.id shown in the visual explains filtering; it is not an output column of this SELECT. An unmatched right side supplies NULL for p.id even though actual stored patient primary keys are non-null.

Follow-up: change IS NOT NULL to IS NULL. Which doctor remains? Ron's patient id is 4; his doctor_id is NULL. These are different columns.

4C. ON filters matches; WHERE filters joined rows

Requirement: show every doctor and only patients aged at least 30. Which query meets it, and what happens to Dr Strange?

Query A:

```
SELECT d.name AS doctor, p.name AS patient
FROM doctors d
LEFT JOIN patients p
    ON p.doctor_id = d.id AND p.age >= 30
ORDER BY d.id, p.id;
```

Query B:

```
SELECT d.name AS doctor, p.name AS patient
FROM doctors d
LEFT JOIN patients p ON p.doctor_id = d.id
WHERE p.age >= 30
ORDER BY d.id, p.id;
```

4C — Put the condition in ON or WHERE?

Same seed, same LEFT JOIN, different treatment of unmatched doctors.

Only include patients aged 30 or above.

A: ON relationship AND p.age >= 30

doctor	patient
Dr Watson	Jean
Dr Banner	Jean
Dr Banner	Luna
Dr Strange	NULL

B: WHERE p.age >= 30

doctor	patient
Dr Watson	Jean
Dr Banner	Jean
Dr Banner	Luna

A keeps Dr Strange as an unmatched left row. B removes his NULL patient-age row after joining. James is excluded by the age condition in both queries.

A preserves the unmatched doctor. B removes the NULL-age row in WHERE. Filtering rows does not create replacement NULL rows after matched rows are removed.

4D. A valid JOIN can answer the wrong question

This condition checks only the doctor ID. Does it connect a patient to their assigned doctor? Predict the row count.

```
SELECT p.id AS patient_id, p.name AS patient, d.id AS doctor_id
FROM patients p
INNER JOIN doctors d ON d.id IN (1, 2)
ORDER BY p.id, d.id;
```

4D — Valid SQL, wrong relationship

ON d.id IN (1, 2) never checks p.doctor_id.

Seeded doctors

id	doctor
1	Dr Watson
2	Dr Banner
3	Dr Strange

Seeded patients

id	patient	age	doctor_id
1	James	20	1
2	Jean	30	1
3	Jean	40	2
4	Ron	17	NULL
5	Luna	30	2
6	John	60	NULL

Every one of 6 patients matches both qualifying doctors: 6 × 2 = 12 rows.

Actual query output — including unassigned patients

patient_id	patient	doctor_id
1	James	1
1	James	2
2	Jean	1
2	Jean	2
3	Jean	1
3	Jean	2
4	Ron	1
4	Ron	2
5	Luna	1
5	Luna	2
6	John	1
6	John	2

To show actual assignments, use ON p.doctor_id = d.id instead; that returns the four assigned patient/doctor pairs.

Six patients each match two qualifying doctors, producing twelve pairs. Replacing ON with p.doctor_id = d.id returns the four actual assignments. Foreign-key constraints do not automatically choose JOIN conditions.

4E. Self-join: one table, two roles

Employee and manager names can be stored in one table. How can aliases represent both roles? The temporary table below is separate from Hospital tables and exists for the current connection. Run its setup once per session.

```
CREATE TEMPORARY TABLE employees_join_demo (
    id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    manager_id INT NULL
);

INSERT INTO employees_join_demo (id, name, manager_id)
VALUES (1, 'Hermione', NULL),
       (2, 'Harry', 1),
       (3, 'Ron', 1),
       (4, 'Neville', 2);
```

```
SELECT e.name AS employee, m.name AS manager
FROM employees_join_demo e
INNER JOIN employees_join_demo m ON e.manager_id = m.id
ORDER BY e.id;
```

4E — Self-join: one table, two roles

e represents an employee; m represents that employee's manager.

Separate temporary employees_join_demo table

id	name	manager_id
1	Hermione	NULL
2	Harry	1
3	Ron	1
4	Neville	2

INNER JOIN output

employee	manager
Harry	Hermione
Ron	Hermione
Neville	Harry

ON e.manager_id = m.id: Harry.manager_id 1 → Hermione.id 1; Neville.manager_id 2 → Harry.id 2.

Follow-up LEFT JOIN: preserve Hermione too

employee	manager
Hermione	NULL
Harry	Hermione
Ron	Hermione
Neville	Harry

ON e.manager_id IS NOT NULL does not identify a manager: it pairs 3 qualifying employees with all 4 possible manager rows, producing 12 rows.

e is the employee role; m is the manager role. e.manager_id = m.id identifies the specific manager. The condition e.manager_id IS NOT NULL alone would match three qualifying employees with every possible manager row, producing twelve rows. LEFT JOIN also retains Hermione, with a NULL manager.

Optional JOIN practice — existing Hospital seed

Use the doctors and six patients seeded earlier. No new rows, schema changes or aggregation are required. These are independent SELECT problems; stored data must remain unchanged. The dataset must be in its original state before any balancing operation.

Problem 1. Pairs of patients assigned to the same doctor

List every pair of different patients assigned to the same doctor. Return the doctor ID/name and both patient IDs/names. Show each pair once: the lower patient ID must appear first, and the reversed pair must not appear again. Unassigned patients must not be paired together. Sort by doctor ID, then the two patient IDs.

Problem 2. Alternative doctor suggestions

For every patient, list the doctors to whom the patient could be reassigned. An assigned patient must not be paired with their current doctor. An unassigned patient may be paired with any existing doctor. Return patient ID/name and suggested doctor ID/name. Sort by patient ID, then suggested doctor ID. This is a report only; no assignment should change.

Problem 3. Doctors with no patients aged 40 or above

Return the ID/name of every doctor who has no assigned patient aged 40 or above. Include doctors with no patients at all. A patient aged exactly 40 counts as 40 or above. Return each qualifying doctor only once, ordered by doctor ID.

5. Aggregation: summarise rows

COUNT(*) counts rows. COUNT(column) counts non-null values. MIN/MAX find the smallest/largest value, AVG computes an average and SUM adds numeric values.

```
SELECT COUNT(*) AS total_patients,
       COUNT(doctor_id) AS assigned_patients,
       MIN(age) AS youngest,
       MAX(age) AS oldest,
       AVG(age) AS average_age
FROM patients;
```

For the seed: total patients 6, assigned patients 4, youngest 17, oldest 60, average approximately 32.83. The two unassigned doctor_id values are excluded from COUNT(doctor_id).

GROUP BY: a report per doctor

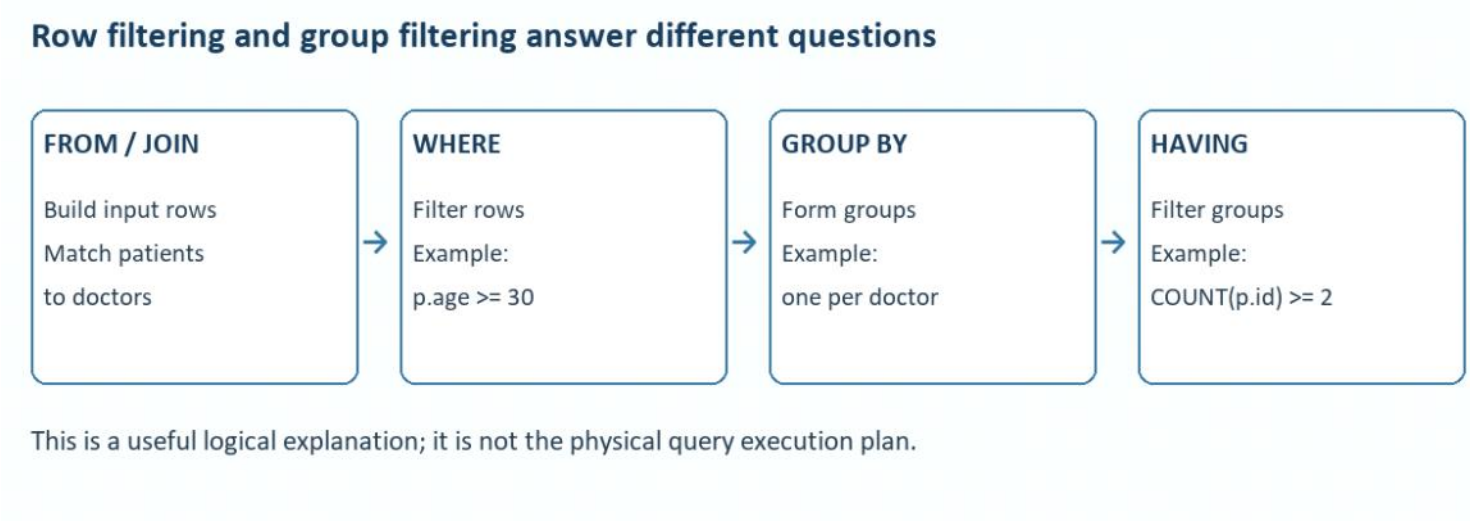
How can all doctors appear, including a doctor with zero patients? Why is COUNT(p.id) different from COUNT(*) after a LEFT JOIN?

```
SELECT d.id, d.name, COUNT(p.id) AS patient_count
FROM doctors d
LEFT JOIN patients p ON p.doctor_id = d.id
GROUP BY d.id, d.name
ORDER BY d.id;
```

The result is Dr Watson 2, Dr Banner 2, Dr Strange 0. The unmatched doctor's joined row exists, but its p.id is NULL; COUNT(p.id) correctly returns zero. Grouping by ID keeps different doctors separate even when their names are identical.

HAVING versus WHERE

```
SELECT d.id, d.name, COUNT(p.id) AS patient_count
FROM doctors d
LEFT JOIN patients p ON p.doctor_id = d.id
GROUP BY d.id, d.name
HAVING COUNT(p.id) >= 2
ORDER BY d.id;
```



WHERE filters input rows; HAVING filters grouped results. The HAVING query retains doctors 1 and 2. To count only patients aged at least 30 per assigned doctor, apply WHERE p.age >= 30 before GROUP BY; the counts become doctor 1:1 and doctor 2:2.

6. Subqueries: use one query's result in another

Which patients are older than the overall average? The inner query computes one average; the outer query compares each patient's age to that value.

```
SELECT id, name, age
FROM patients
WHERE age > (SELECT AVG(age) FROM patients)
ORDER BY id;
```

Result: Jean/3, age 40, and John/6, age 60. This is a logical explanation of the expression, not a guarantee about physical database execution order.

Second-highest distinct age

```
SELECT MAX(age) AS second_highest_age
FROM patients
WHERE age < (SELECT MAX(age) FROM patients);
```

Result: 40. Excluding the maximum removes every row tied at the highest age. With fewer than two distinct ages, the result is NULL. The second-highest distinct value is not necessarily the second sorted row.

7. Connect the existing Spring application to MySQL

Keep spring-boot-starter-jdbc. Replace the H2 runtime dependency with the MySQL driver and remove the H2 console dependency from this MySQL configuration. The existing Spring Boot parent manages the driver version.

```
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
```

JdbcTemplate is the Spring helper; Connector/J supplies the MySQL-specific JDBC driver. The existing services and controllers continue using their current dependencies.

Set MYSQL_USER and MYSQL_PASSWORD privately in the IntelliJ application's Run Configuration. Use the same tested MySQL account as the SQL-client connection. Configure application.properties:

```
spring.application.name=springProject
server.port=8081

spring.datasource.url=jdbc:mysql://localhost:3306/hospital_lecture12
spring.datasource.username=${MYSQL_USER}
spring.datasource.password=${MYSQL_PASSWORD}

spring.sql.init.mode=never
```

localhost:3306 identifies the database server; 8081 is the application's HTTP port. The database and tables must already exist. The JDBC URL selects the database; it does not copy H2 rows. init.mode=never prevents old H2 sequence/MERGE initialization scripts from running against MySQL. This application still uses JdbcTemplate, not JPA.

8. Use AUTO_INCREMENT instead of the H2 sequence

Replace only the existing add method in HospitalPatientRepository. Keep its existing mapper, constructor and other methods.

```
public void add(PatientInput input) {
    jdbc.update(
            "INSERT INTO patients (name, age) VALUES (?, ?)",
            input.getName(), input.getAge());
}
```

Omitting id lets MySQL allocate it; omitting nullable doctor_id creates an unassigned patient. The current API returns the resulting patient list, so this operation does not need a separate generated-key lookup. IDs are unique identifiers, not gap-free counters; never allocate them using collection size + 1 or MAX(id) + 1.

DoctorRepository remains under Doctors.Repository and retains assignPatients, doctorRowMapper, patientRowMapper and the ordered patient query. Its standard SELECT/UPDATE statements continue working with MySQL.

9. Verify MySQL persistence and the homework endpoints

Use Workbench to inspect the MySQL database; the old H2 Console is not a viewer for this connection.

```
GET /hospital/patients
→ the six seeded MySQL patients

POST /hospital/addPatient
{"name":"Frodo","age":26}
→ 200, list includes a generated patient ID

PUT /hospital/updatePatient/{frodoId}
{"name":"Frodo Baggins","age":99}
→ name becomes Frodo Baggins; stored age remains 26

PUT /doctors/1/patients/{frodoId}
GET /doctors/1/patients
→ includes the current Frodo Baggins row

Restart Spring and repeat GET
→ rows and assignments remain in MySQL

DELETE /hospital/deletePatients/{frodoId}
→ 200, remaining list

Create another patient
→ generated ID does not overwrite a stored patient
```

```
SELECT id, name, age, doctor_id FROM patients ORDER BY id;
```

The pushed Hospital update endpoint changes the patient's name only. Its @Valid request handling rejects an age of 121 before the service runs; verify a 400 and unchanged stored name/age. The current primitive age field does not distinguish an omitted value from zero.

Workload after the MySQL switch

After removing only the temporary API-created patients, the original seed has doctor counts 2/2/0. Predict the source, target and patient selected for one balancing step.

```
GET /doctors/workload → counts 2, 2, 0
POST /doctors/rebalance → patient 1 moves from doctor 1 to doctor 3
GET /doctors/workload → doctor 2:2, doctor 1:1, doctor 3:1
```

Restore the seed assignment with PUT /doctors/1/patients/1 and verify patient 1 has doctor_id 1 before repeating the original SQL exercises. No table reset or reseeding is required.

10. SQL practice

Write queries for: unassigned patients; every patient with doctor name when present; doctors without patients; patient names appearing more than once; second-highest distinct age; counts for every doctor including zero.

Questions to explain: why does an outer join help preserve missing matches? When does WHERE change that preservation? Which result needs HAVING? Why can COUNT(*) give the wrong zero-patient count after LEFT JOIN?